# orthoresponse

Expression response models of orthologs and genes in two-by-two designs

User manual • Version 0.4.1 • Python and R



stress-ready shared unique opposite no response

Two groups (species, genotypes, cultivars or cell lines) × two conditions (control and stress). For every gene or ortholog pair: does one group already express, without stress, what the other only reaches under stress?

**What you will find in this manual**

- What the five response models mean
- How the decision tree works
- Installation in Python and R
- Quick start with bundled data
- Input formats and templates
- Options, presets and their effect
- Every output file and figure
- Two worked examples with real data
- Cross-species analyses with orthologs
- Interpretation caveats and troubleshooting

J. Sebastián Contreras-Riquelme

github.com/Cold7/orthoresponse • MIT license • November 2026

# Contents

**1 What is orthoresponse? 1**
1.1 Key features . . . 1
1.2 When to use it . . . 1

**2 Concepts 2**
2.1 The two-by-two design . . . 2
2.2 The fifteen patterns . . . 2

**3 How it works 4**
3.1 The workflow . . . 4
3.2 The decision tree . . . 4

**4 Installation 5**
4.1 Python (3.9 or newer) . . . 5
4.2 R (4.1 or newer) . . . 5

**5 Quick start 5**
5.1 Run the bundled example . . . 5
5.2 Run it on your own data . . . 6
5.3 Command line . . . 7

**6 Input data 7**
6.1 Count matrix . . . 7
6.2 Sample table . . . 7
6.3 Precomputed contrasts . . . 8

**7 Presets and options 8**

**8 Outputs 9**
8.1 Files . . . 9
8.2 The classification table . . . 10
8.3 Summaries and denominators . . . 10
8.4 Figures . . . 10

**9 Worked examples 11**
9.1 Plant: rice under salt stress (GEO GSE167342) . . . 11
9.2 Human: ovarian cancer cells and brequinar (GEO GSE283381) . . . 13

**10 Interpreting results 14**

**11 Cross-species analyses 15**

**12 Python and R 16**

**13 Function reference 16**

**14 Troubleshooting 17**

**15 Citation and license 17**

# 1 What is orthoresponse?

**orthoresponse** is a Python and R package that compares how *two groups* respond to *the same stress*. The groups can be two species (compared through their orthologous genes), two genotypes or cultivars of one species, or two cell lines. Each group is measured under a **control** and a **stress** condition, so the experiment is a $2 \times 2$ design.

For every gene, or every pair of orthologs, the package decides which of five *expression response models* describes the data best:

| | |
|---|---|
| **stress-ready** | one group responds, and under stress it reaches the level that the other group already has *without* stress. The other group is *primed*. |
| **shared** | both groups respond in the same direction. |
| **unique** | only one group responds. |
| **opposite** | both groups respond, in opposite directions. |
| **no response** | neither group responds (the basal levels may still differ). |

The idea of a *stress-ready* (or pre-adapted) transcriptome comes from comparisons between stress-sensitive plants and their stress-tolerant relatives, where the tolerant species constitutively express genes that the sensitive species only induce under stress. With its default settings **orthoresponse** reproduces exactly the decision tree used by Contreras-Riquelme *et al.* (2025, *PLoS One* 20(5): e0324724) to compare cultivated tomato and the desert-adapted *Solanum pennellii* under drought, and adds stricter, optional rules.

## 1.1 Key features

- **One call from counts to answers**: a single DESeq2 (R) or PyDESeq2 (Python) fit, five contrasts, classification, summaries and figures.
- **Two presets**: `paper` (the published rules) and `strict` (handles missing statistics and requires a basal difference for stress-ready calls), plus an optional formal equivalence test.
- **Identical in Python and R**: same function names, options and outputs; both implementations give the same label for every feature given the same contrasts.
- **Cross-species support**: reciprocal best BLAST hits and ortholog-level count matrices.
- **Real example data included**: rice under salt stress and human ovarian cancer cells under a metabolic drug, ready to run.
- **Publication-ready figures** saved automatically (PNG and PDF), with configurable labels.

## 1.2 When to use it

Use **orthoresponse** when you have RNA-seq raw counts for a complete $2 \times 2$ design and want to know whether one group is *pre-adapted* to a stress, which responses are conserved, and which are group-specific. Typical questions:

- Does a drought-tolerant wild relative constitutively express genes that the crop only induces under drought?
- Does a drug-resistant cell line already express what the sensitive line induces under the drug?

- Which stress responses are shared by two cultivars, and which are unique to the tolerant one?

# 2 Concepts

## 2.1 The two-by-two design

Every analysis is built on four groups of samples and five pairwise comparisons (Figure 1). Group **A** is the *reference*, usually the sensitive group; group **B** is the *comparison*, usually the tolerant or adapted one. Which group you call **A** only changes the case numbers, never the biology.

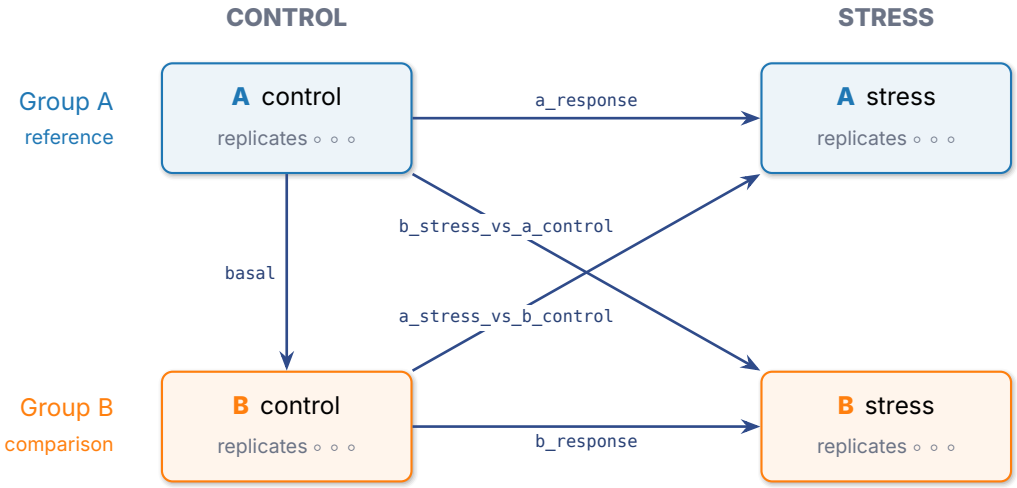



**Figure 1:** The $2 \times 2$ design and the five contrasts used by the classifier. Each arrow points from the denominator to the numerator: a positive log2 fold change means *higher at the arrow head*.

## 2.2 The fifteen patterns

The five models split into fifteen cases according to the direction of the change (Figure 2, Table 1). In every panel the left half shows group **A** and the right half group **B**; within each half, control is on the left and stress on the right.

> **In short** Stress-ready cases 1–2 mean that **A** is primed; cases 3–4 mean that **B** is primed. If **B** is your tolerant group, cases 3 and 4 are the ones that support pre-adaptation of **B**.

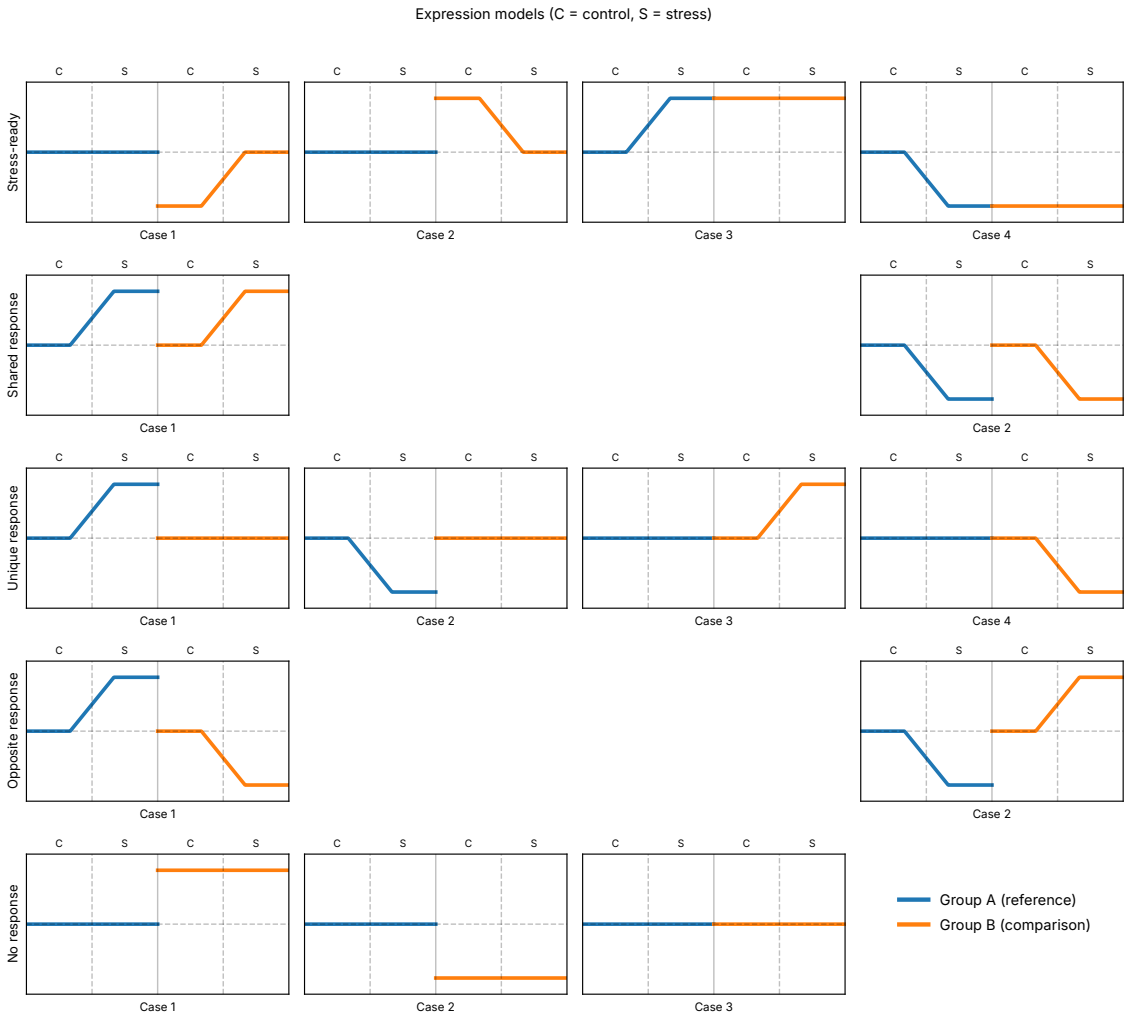



**Figure 2:** Schematic of the fifteen expression patterns (C = control, S = stress). Shared and opposite responses are drawn in the outer columns.

**Table 1:** Labels produced by the classifier. The *primed* group is the one whose control level equals the other group's stressed level.

| label | meaning | primed |
|---|---|---|
| `stress_ready_1` | B is induced under stress up to the control level of A | A |
| `stress_ready_2` | B is repressed under stress down to the control level of A | A |
| `stress_ready_3` | A is induced under stress up to the control level of B | B |
| `stress_ready_4` | A is repressed under stress down to the control level of B | B |
| `shared_1 / _2` | induced / repressed in both groups | – |
| `unique_1 / _2` | induced / repressed only in A | – |
| `unique_3 / _4` | induced / repressed only in B | – |
| `opposite_1 / _2` | induced in A and repressed in B / the reverse | – |
| `no_response_1 / _2` | no response; basal level higher / lower in B than in A | – |
| `no_response_3` | no response; no basal difference detected | – |
| `unresolved_`$k$ | only with an equivalence test: neither different nor equivalent | – |
| `unclassified` | only with `na_action="unclassified"`: missing statistics | – |

# 3 How it works

## 3.1 The workflow

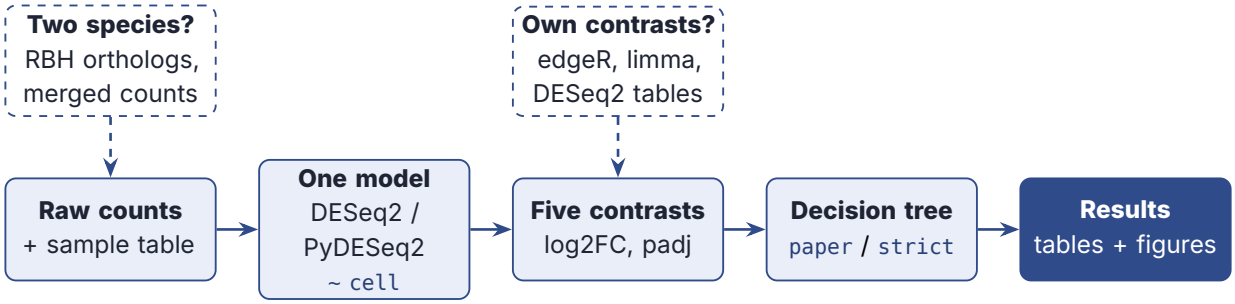



**Figure 3:** From counts to classification. Dashed boxes are optional entry points.

1. **Design.** Samples are mapped to four cells: `A_control`, `A_stress`, `B_control`, `B_stress`.
2. **Filtering.** Features with a total count $\leq$ `min_total_count` (default 10) are removed.
3. **Model.** One negative-binomial model is fitted to all samples with the design `~ cell`; size factors and dispersions are estimated across both groups.
4. **Contrasts.** The five contrasts of Figure 1 are extracted with Benjamini–Hochberg adjustment, Cook's outlier filtering and independent filtering.
5. **Classification** with the decision tree below.
6. **Summaries and figures**: counts per model with explicit denominators, the primed-asymmetry test, and the figures of Section 8.

## 3.2 The decision tree

A contrast is *significant* when its adjusted p-value is below `alpha` (default 0.05) and, optionally, $|\log_2 \mathrm{FC}|$ is at least `min_abs_lfc`. Its *direction* is the sign of the log2 fold change.

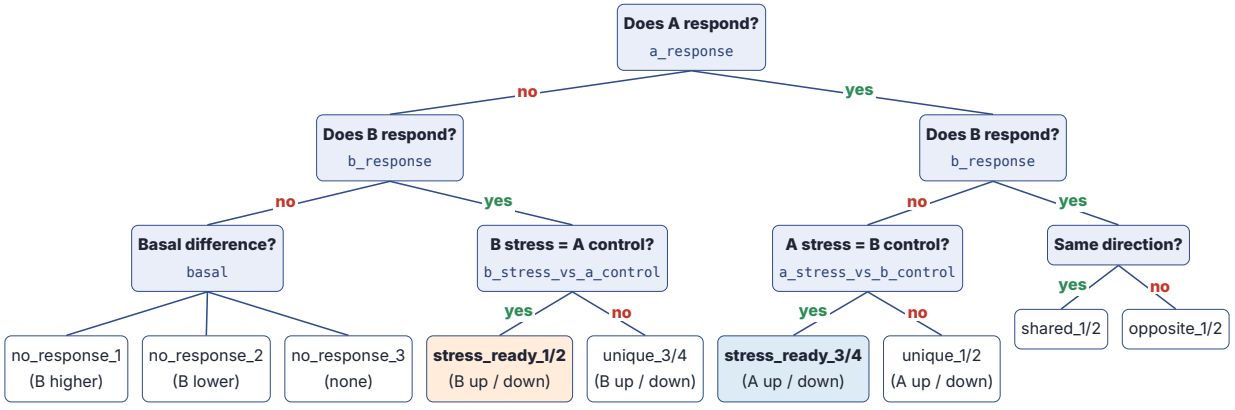



**Figure 4:** The decision tree. "Same level" (=) means *not significantly different* under the `paper` preset; Section 7 describes stricter alternatives.

**Absence of evidence is not evidence of equality** Under the `paper` preset a gene is stress-ready when the responding group's stressed level is *not significantly different* from the other group's control level. With few replicates this inflates the stress-ready class. The `strict` preset and the equivalence test exist for this reason; see Sections 7 and 10.

# 4 Installation

## 4.1 Python (3.9 or newer)

Shell

```
# full installation: PyDESeq2, plotting and Excel support
pip install "orthoresponse[all] @ \
  git+https://github.com/Cold7/orthoresponse.git#subdirectory=python"

# a specific version
pip install "orthoresponse[all] @ \
  git+https://github.com/Cold7/orthoresponse.git@v0.4.1#subdirectory=python"

orthoresponse --version
```

Optional extras: `deseq` (PyDESeq2), `plot` (matplotlib), `excel` (openpyxl), `all`, `test`. Without extras only the classifier (numpy, pandas, scipy) is installed, which is enough if you already have contrast tables. Installing from GitHub requires `git`.

## 4.2 R (4.1 or newer)

R

```
install.packages(c("BiocManager", "remotes"))
BiocManager::install("DESeq2")
remotes::install_github("Cold7/orthoresponse", subdir = "r-package",
                        build_vignettes = TRUE)
library(orthoresponse)
vignette("orthoresponse")
```

# 5 Quick start

## 5.1 Run the bundled example

The package includes two real data sets (Section 9). This is the fastest way to see the whole workflow:

Python

```
import orthoresponse as orr

ex = orr.load_example("rice_salt")            # ex.counts, ex.samples, ex.params
res = orr.run_orthoresponse(ex.counts, ex.samples, **ex.params)
print(res)                                    # summary + asymmetry test
res.save("results")                           # tables + figures
```

R

```r
library(orthoresponse)
ex  <- load_example("rice_salt")
res <- do.call(orthoresponse, c(list(ex$counts, ex$samples), ex$params))
res                                          # same output as print(res)
save_orthoresponse(res, "results")           # tables + figures
```

Both languages print the same text (shown here for R; Python's `print(res)` is identical in format):

R console

```
orthoresponse result: A = PL12 | B = IL22
25045 features classified

          label        model case     n pct_of_responsive pct_of_all
 stress_ready_1 stress_ready    1   281              4.1        1.1
 stress_ready_2 stress_ready    2   407              6.0        1.6
 stress_ready_3 stress_ready    3  1744             25.5        7.0
 stress_ready_4 stress_ready    4  1793             26.2        7.2
       shared_1       shared    1   563              8.2        2.2
       shared_2       shared    2   335              4.9        1.3
       unique_1       unique    1   617              9.0        2.5
       unique_2       unique    2   628              9.2        2.5
       unique_3       unique    3   152              2.2        0.6
       unique_4       unique    4   185              2.7        0.7
     opposite_1     opposite    1    88              1.3        0.4
     opposite_2     opposite    2    41              0.6        0.2
  no_response_1  no_response    1   174               NA        0.7
  no_response_2  no_response    2   196               NA        0.8
  no_response_3  no_response    3 17841               NA       71.2

Stress-ready among single-group responders: PL12-only 74.0%, IL22-only 67.1% (Fisher p = 1.22e-05)
```

## 5.2 Run it on your own data

Python

```python
import pandas as pd
import orthoresponse as orr

counts  = pd.read_csv("counts.tsv",  sep="\t", index_col=0)  # raw counts
samples = pd.read_csv("samples.tsv", sep="\t", index_col=0)  # 1 row/sample

res = orr.run_orthoresponse(
    counts, samples,
    group_col="line", condition_col="treatment",
    group_a="PL12", group_b="IL22",        # A = reference, B = comparison
    control="Control", stress="Salt")
res.save("results", condition_labels=("Ctrl", "NaCl"))
```

R

```r
counts  <- read.delim("counts.tsv",  row.names = 1, check.names = FALSE)
samples <- read.delim("samples.tsv", row.names = 1)
res <- orthoresponse(counts, samples,
                     group_col = "line", condition_col = "treatment",
                     group_a = "PL12", group_b = "IL22",
                     control = "Control", stress = "Salt")
save_orthoresponse(res, "results", condition_labels = c("Ctrl", "NaCl"))
```

**The four values must match your sample table** `group_a`, `group_b`, `control` and `stress` are compared literally (and case-sensitively) with the values in `group_col` and `condition_col`. If none match, the package stops with *Empty design cell*(s).

### 5.3 Command line

The Python package installs the `orthoresponse` command:

Terminal

```
$ orthoresponse example --out my_example
Wrote my_example/counts.tsv.gz and my_example/samples.tsv
Run it with:
  orthoresponse run --counts my_example/counts.tsv.gz --samples my_example/samples.tsv
  --group-col line --condition-col treatment --group-a PL12 --group-b IL22
  --control Control --stress Salt --out results

$ orthoresponse run --counts my_example/counts.tsv.gz --samples my_example/samples.tsv \
    --group-col line --condition-col treatment --group-a PL12 --group-b IL22 \
    --control Control --stress Salt --condition-labels Ctrl NaCl --out results

$ orthoresponse models --group-names PL12 IL22      # definitions of every case
$ orthoresponse classify --help                     # classify precomputed contrasts
```

## 6 Input data

### 6.1 Count matrix

- **Raw integer counts** – not TPM, FPKM or normalized values. Genes (or ortholog pairs) in rows, samples in columns, identifiers in the first column.
- Estimated counts from Salmon or Kallisto are accepted with `round_counts=True`.
- Tab- or comma-separated, optionally gzip-compressed; Excel files can be read with pandas or readxl.

### 6.2 Sample table

One row per sample, with the sample names exactly as in the count matrix columns, and at least one column for the group and one for the condition. Samples with other values are ignored, so a single table can describe a larger experiment.

| sample | line | treatment | geo_sample |
|---|---|---|---|
| S40 | PL12 | Control | GSM5102551 |
| S52 | IL22 | Control | GSM5102554 |
| S58 | PL12 | Salt | GSM5102557 |
| S70 | IL22 | Salt | GSM5102560 |
| ⋮ 12 rows in total | | | |

**Use real tabs** Copy-pasting a table often turns tabs into spaces, which produces errors such as *Expected 2 fields in line 3, saw 3*. Save the file from a spreadsheet as tab-separated text, or read it with `sep=r"\s+"`. To start from a correct template run `orthoresponse example --out my_folder` or `export_example("my_folder")`.

Use at least 3 replicates per cell; with 2, treat gene-level calls as exploratory.

### 6.3 Precomputed contrasts

If you already have differential-expression results, call `classify_responses()` directly with five tables. Each needs `log2FoldChange` and `padj` (and `lfcSE` for the equivalence test):

Python

```
tabs = orr.read_contrasts({
    "a_response":            "A_stress_vs_A_control.csv",
    "b_response":            "B_stress_vs_B_control.csv",
    "basal":                 "B_control_vs_A_control.csv",
    "b_stress_vs_a_control": "B_stress_vs_A_control.csv",
    "a_stress_vs_b_control": "A_stress_vs_B_control.csv",
})
cls = orr.classify_responses(tabs, group_names=("sensitive", "tolerant"))
```

Column names from other tools can be mapped with `lfc_col`, `padj_col` and `se_col`. The sign convention matters: a positive log2 fold change must mean *higher in the numerator*.

## 7 Presets and options

**Table 2:** Classification options. Any option can be combined with any preset.

| option | paper | strict | meaning |
|---|---|---|---|
| `alpha` | 0.05 | 0.05 | adjusted p-value threshold |
| `min_abs_lfc` | 0 | 0 | minimum $\lvert \log_2 \text{FC} \rvert$ for a significant change |
| `na_action` | `not_de` | `unclassified` | how a missing `padj` is treated |
| `require_basal_difference` | no | yes | stress-ready needs a significant basal difference |
| `equivalence_margin` | none | none | if set, “same level” must pass a TOST equivalence test |
| `sign_digits` | 2 | 2 | rounding of log2FC before taking its sign |

Python

```
orr.classify_responses(tabs, preset="strict")
orr.classify_responses(tabs, preset="paper", equivalence_margin=0.5, alpha=0.01)
```

### Missing adjusted p-values

DESeq2 sets `padj` to missing for low-count genes (independent filtering) and outliers. Under `paper` a missing value counts as "not significant", which pushes features towards stress-ready; the `flags` column marks them with `na_treated_as_not_de`. With `na_action="unclassified"` they are labelled `unclassified` instead.

### Basal difference

If A is induced up to B's control level (stress-ready 3), B's control level must be higher than A's. The `paper` rules never check this. With `require_basal_difference=True`, a stress-ready call also needs a significant `basal` contrast in the expected direction; otherwise the feature becomes the corresponding unique response and is flagged `basal_difference_not_significant`.

### Equivalence test

With `equivalence_margin` $= m$ (log2 units), the matching contrast must show $|\log_2 \mathrm{FC}| < m$ through two one-sided Wald tests:

$$p_{\mathrm{TOST}} = \max\left\{ P\left(Z > \frac{\hat{\beta} + m}{\mathrm{SE}}\right),\ P\left(Z < \frac{\hat{\beta} - m}{\mathrm{SE}}\right) \right\},$$

adjusted with Benjamini–Hochberg across features. This is the construction DESeq2 uses with `altHypothesis = "lessAbs"`. Features that are neither different nor equivalent are labelled `unresolved`. Choose $m$ before looking at the data; 0.5–1 is common.

> **Which preset?** Use `paper` to reproduce or compare with the original publication. Use `strict`, ideally with an equivalence margin, when you want to claim that a group is pre-adapted, and always when the two groups are near-isogenic. Reporting both is the most informative choice.

## 8 Outputs

### 8.1 Files

`res.save("results")` (Python) and `save_orthoresponse(res, "results")` (R) write the files of Figure 5.

```
results/
└─ classification.tsv                one row per gene: label, model, case, groups, flags, statistics
└─ summary.tsv                       counts and percentages per model and case
└─ primed_asymmetry.tsv              direction test for stress-ready states
└─ normalized_counts.tsv             size-factor normalized counts
└─ contrasts/contrast_<key>.tsv      full DESeq2 tables of the five contrasts
└─ model_patterns.png / .pdf         the fifteen patterns with counts and percentages
└─ model_summary.png / .pdf          bar chart of genes per model and case
```

Figures: `plots=False` turns them off; `condition_labels`, `model_labels`, `group_names`, `title` and `formats` customize them.

**Figure 5:** Contents of the results folder.

## 8.2 The classification table

| column | content |
|---|---|
| `label` | e.g. `stress_ready_3` |
| `model, case` | the two parts of the label |
| `primed_group` | group whose control level equals the other's stressed level (stress-ready only) |
| `responding_group` | A, B, `both` or `none` (with your group names) |
| `description` | a readable sentence, e.g. "PL12 is induced under stress up to the control level of IL22" |
| `flags` | `na_treated_as_not_de`, `basal_difference_not_significant` |
| `<key>_log2FoldChange, <key>_padj` | statistics of the five contrasts |

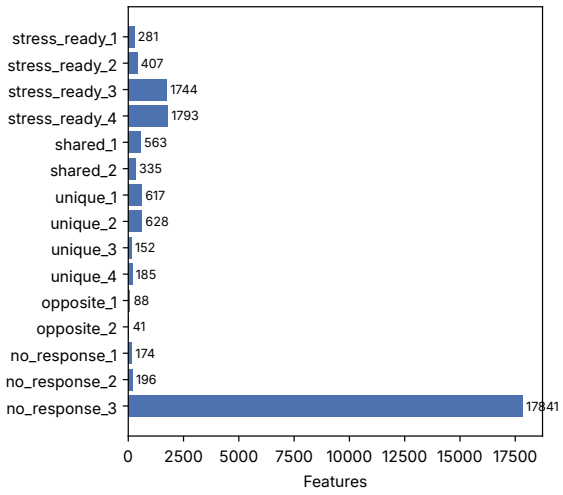
## 8.3 Summaries and denominators

`summary` reports two percentages on purpose: `pct_of_responsive` uses the genes that respond in at least one group, and `pct_of_all` uses every classified gene. Mixing the two denominators is a common source of confusion when reporting results.

`primed_asymmetry()` asks whether one group is *more often* primed. A larger raw number of stress-ready genes in one direction can simply mean that one group has more responsive genes; the test compares the *fraction* of single-group responses that end in a stress-ready state, with Fisher's exact test.

## 8.4 Figures

Two figures are written next to the tables: `model_summary`, a bar chart of genes per model and case, and `model_patterns`, the fifteen patterns annotated with counts and percentages.




**model_summary** – genes per model and case (left, rice example).

**model_patterns** – the fifteen patterns annotated with counts and percentages (Figures 7 and 8).

Customize them:

```
res.save("results",
  condition_labels=("Ctrl", "NaCl"),
  model_labels={"stress_ready": "Primed"},
  formats=("png",))
```

**Figure 6:** Bar chart written as `model_summary.png/pdf`.

# 9 Worked examples

Both data sets are bundled with the package and come from raw counts deposited by the authors in GEO; no read alignment is needed. The numbers below come from the R/DESeq2 run.

| name | groups (A vs B) | conditions | size |
|---|---|---|---|
| `rice_salt` | PL12 (salt-sensitive) vs IL22 (*Saltol* introgression line) | control vs 24 h 80 mM NaCl | 36 844 × 12 |
| `ovarian_brequinar` | OVCAR5 cisplatin-sensitive vs cisplatin-resistant | vehicle vs brequinar | 59 453 × 12 |

## 9.1 Plant: rice under salt stress (GEO GSE167342)

Leaves of the salt-sensitive *japonica* cultivar **PL12** (A) and the salt-tolerant introgression line **IL22** (B; FL478 × PL12, carrying the *Saltol* QTL), control vs 24 h of 80 mM NaCl, 3 replicates. Columns S40–S72 of the count matrix are the authors' library names; the bundled sample table links each one to its GEO sample.

Shell

```
cd examples/rice_salt_GSE167342
python run_python.py        # or: Rscript run_r.R
```

**What it shows.**

- The sensitive line reacts much more: 4 782 single-line responses in PL12 against 1 025 in IL22, consistent with IL22 experiencing less stress.
- The lines are nearly isogenic, so basal differences are rare (370 non-responsive genes). Under the `paper` rules 62% of the responsive genes are called stress-ready, mostly because a gene that responds in one line is "not significantly different" from the unchanged level of the other. Requiring a basal difference keeps 593 of 4 225.
- The apparent asymmetry (74.0% vs 67.1%, $p = 1.2 \times 10^{-5}$) disappears under `strict` (10.4% vs 10.6%, $p = 0.82$).

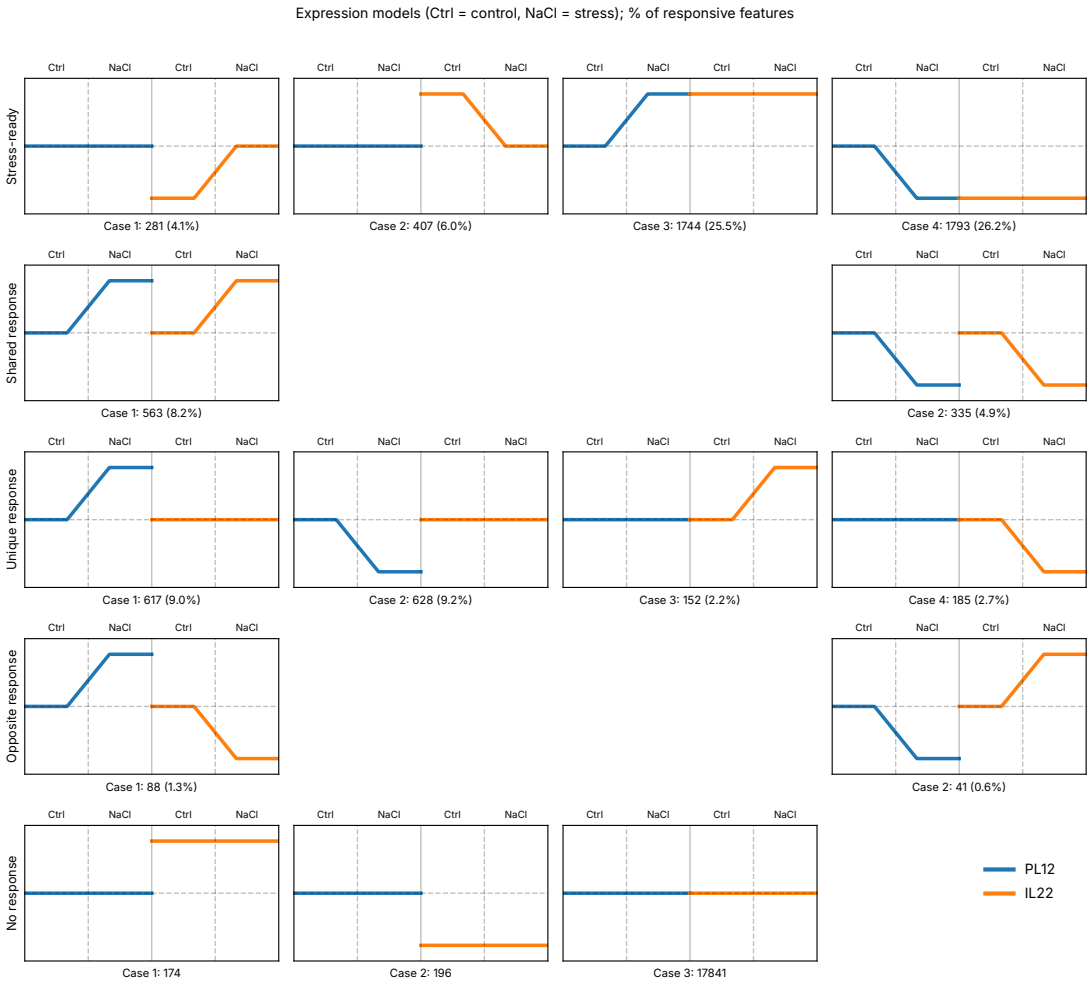



**Figure 7:** Rice example, `paper` preset (Ctrl = control, NaCl = salt). 25 045 genes after filtering.

**Table 3:** Rice: number of genes per model under the two presets.

| model | paper | strict |
|---|---|---|
| stress-ready 1 / 2 / 3 / 4 | 281 / 407 / 1 744 / 1 793 | 35 / 72 / 297 / 189 |
| shared 1 / 2 | 563 / 335 | 563 / 335 |
| unique 1 / 2 / 3 / 4 | 617 / 628 / 152 / 185 | 2 024 / 2 180 / 389 / 514 |
| opposite 1 / 2 | 88 / 41 | 88 / 41 |
| no response 1 / 2 / 3 | 174 / 196 / 17 841 | 174 / 196 / 13 587 |
| unclassified | 0 | 4 361 |

## 9.2 Human: ovarian cancer cells and brequinar (GEO GSE283381)

OVCAR5 high-grade serous ovarian cancer cells: the cisplatin-sensitive parental line **CisS** (A) and a cisplatin-resistant derivative **CisR** (B), treated with vehicle or **brequinar**, an inhibitor of DHODH and therefore of *de novo* pyrimidine synthesis, 3 replicates. The raw HTSeq counts deposited by the authors (one file per sample) were merged into one matrix without any transformation.

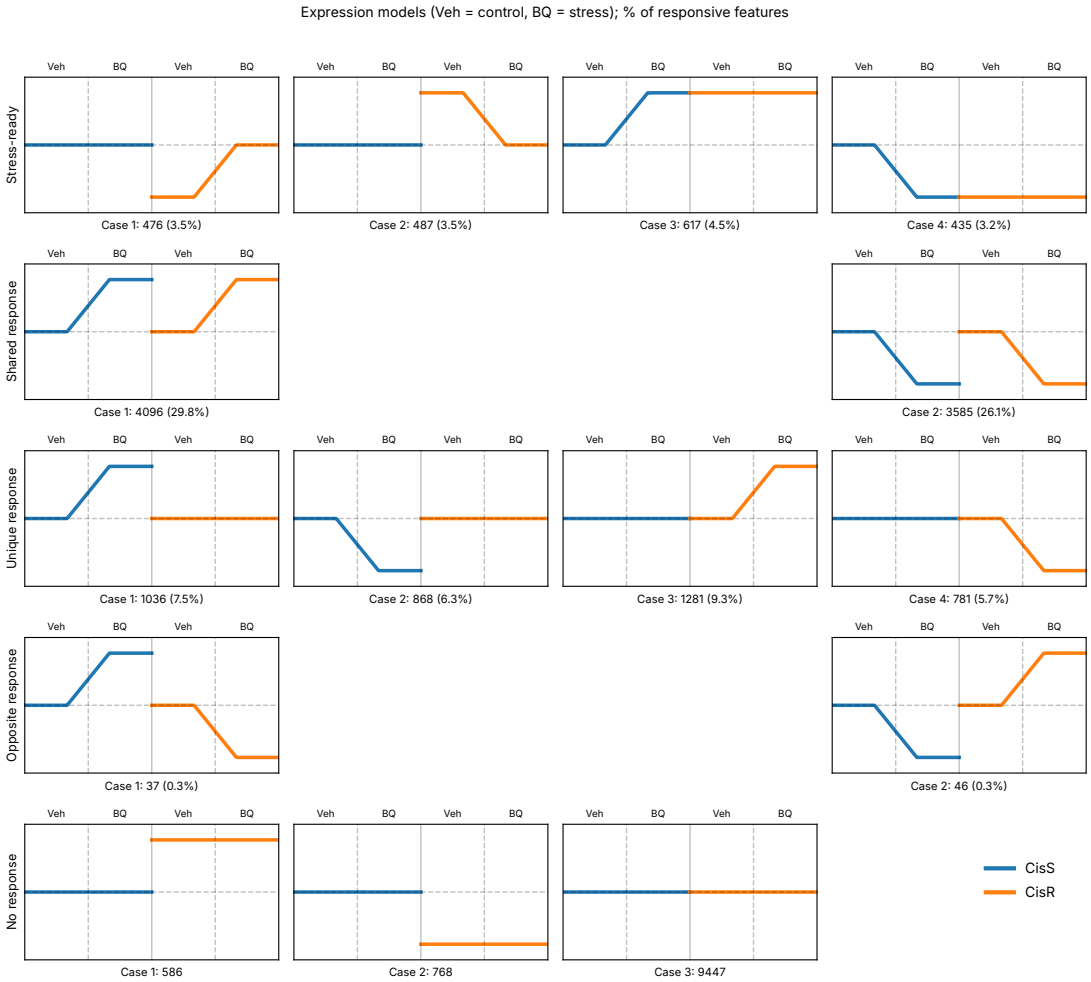



**Figure 8:** Ovarian example, `paper` preset (Veh = vehicle, BQ = brequinar). 24 546 genes after filtering.

**What it shows.**

- Brequinar changes many genes in both lines: most responsive genes are shared (7 681 of 13 745).
- The two cell lines differ at basal level for many genes, so `strict` keeps about a third of the stress-ready calls (646 of 2 015), compared with one in seven in rice.
- Neither line is globally more primed: 35.6% vs 31.8% under `paper` ($p = 0.002$) and 10.8% vs 11.4% under `strict` ($p = 0.43$).

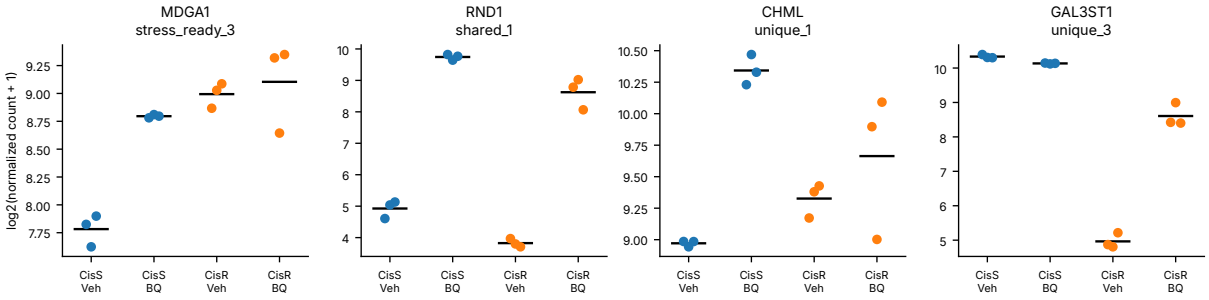



**Figure 9:** Real genes from the ovarian example, one per model (normalized counts; dots are replicates, bars are means). MDGA1: induced by brequinar in the sensitive cells up to the constitutive level of the resistant cells (stress-ready 3, `strict` preset).

**Table 4:** Ovarian cells: number of genes per model under the two presets.

| model | paper | strict |
|---|---|---|
| stress-ready 1 / 2 / 3 / 4 | 476 / 487 / 617 / 435 | 162 / 179 / 159 / 146 |
| shared 1 / 2 | 4 096 / 3 585 | 4 096 / 3 585 |
| unique 1 / 2 / 3 / 4 | 1 036 / 868 / 1 281 / 781 | 1 409 / 1 123 / 1 568 / 1 072 |
| opposite 1 / 2 | 37 / 46 | 37 / 46 |
| no response 1 / 2 / 3 | 586 / 768 / 9 447 | 586 / 768 / 6 755 |
| unclassified | 0 | 2 855 |

## 10 Interpreting results

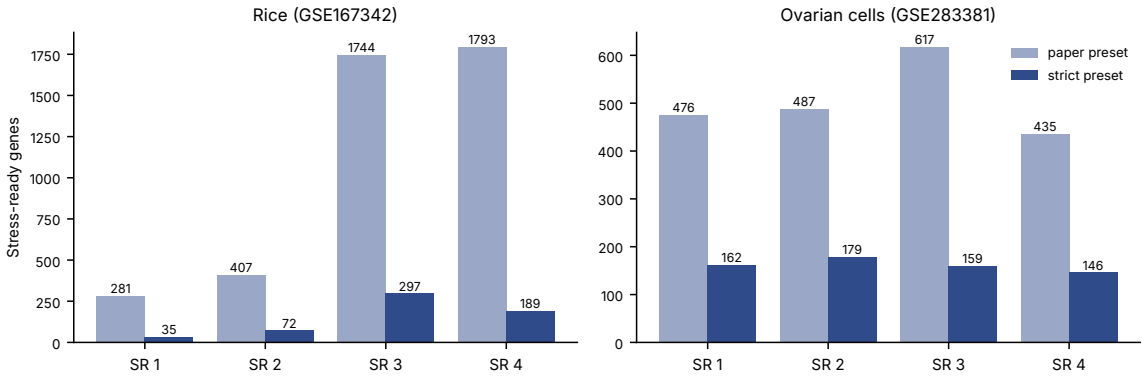



**Figure 10:** Stress-ready genes under the two presets. The reduction is strongest when the groups have few basal differences (rice).

- **False positives flow into stress-ready.** A false-positive response in one group, for a gene whose basal level does not differ between groups, is classified as stress-ready under the `paper` rules. `require_basal_difference` blocks this route.
- **Unique vs shared.** "Significant in A and not in B" does not prove that the responses differ. If the difference of responses is your question, test the interaction (`~ group + group:condition`) separately.
- **Directional claims.** Use `primed_asymmetry()` before concluding that one group is more pre-adapted.

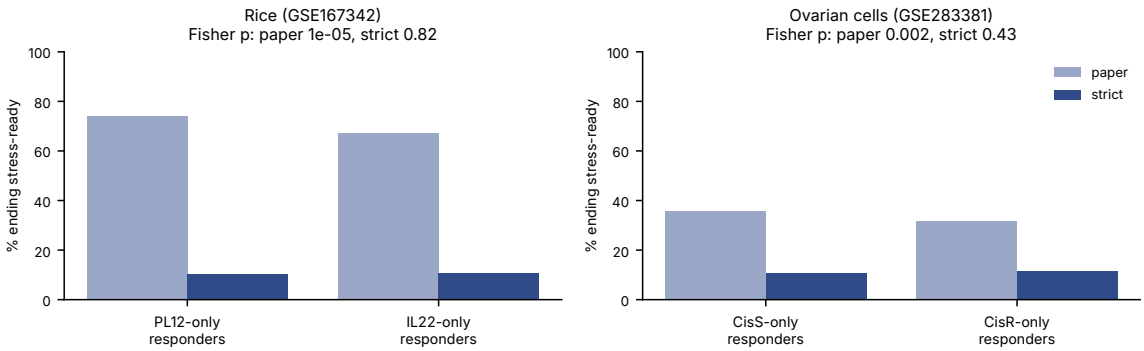



**Figure 11:** Fraction of single-group responses that end in a stress-ready state. Under the `paper` rules a small asymmetry appears in both data sets; under `strict` it disappears.

- **Equal stress?** The same treatment may be perceived differently by the two groups; fewer responses in one group can mean less perceived stress rather than pre-adaptation.
- **Replicates.** At least 3 per cell; with 2, results are exploratory.

## 11 Cross-species analyses

When the two groups are different species, each gene must be paired with its ortholog first.

1. Quantify each species against its own reference to obtain raw gene-level counts.
2. Define one-to-one orthologs, for example as reciprocal best BLASTp hits:

```shell
makeblastdb -in B.faa -dbtype prot -out B
makeblastdb -in A.faa -dbtype prot -out A
blastp -query A.faa -db B -outfmt 6 -evalue 1e-5 -max_target_seqs 5 > A_vs_B.tsv
blastp -query B.faa -db A -outfmt 6 -evalue 1e-5 -max_target_seqs 5 > B_vs_A.tsv
```

3. Build the ortholog table and merge the two count matrices:

```python
orth = orr.reciprocal_best_hits("A_vs_B.tsv", "B_vs_A.tsv",
                                strip_pattern=r"\.\d+$")
counts = orr.merge_ortholog_counts(counts_a, counts_b, orth)
res = orr.run_orthoresponse(
    counts, samples, group_col="species", condition_col="treatment",
    group_a="S_lycopersicum", group_b="S_pennellii",
    control="control", stress="drought")
```

Any other one-to-one ortholog table (OrthoFinder single-copy orthogroups, Ensembl Compara, PLAZA) works as long as it has one column per species. `length_normalize=True` applies the length correction $Q = \mathrm{round}(E/L_{\mathrm{kb}})$ of the original tomato study; use it to replicate that analysis or as a sensitivity check. Remember that basal differences between species mix biology with technical effects such as reference quality, transcript length and annotation.

## 12 Python and R

- **Classifier: identical.** Given the same contrast tables, both languages return the same label for every feature. This was verified on 240 000 random classifications with four option sets, on a shared fixture used by both test suites, and on both worked examples.
- **Differential-expression engine: very close.** DESeq2 and PyDESeq2 are separate implementations. Fold changes agree almost perfectly (correlation $> 0.9999$), but adjusted p-values near the threshold can differ: 99.5% of labels agreed on the rice example and 96.3% on the ovarian one.

**Exact agreement across languages** Compute the contrasts once (e.g. in R), write them with `write_contrasts()`, and classify the same tables in either language.

## 13 Function reference

| Python | R | purpose |
|---|---|---|
| `run_orthoresponse()` | `orthoresponse()` | counts → contrasts → classification |
| `run_contrasts()` | `run_contrasts()` | DESeq2 / PyDESeq2 fit and the five contrasts |
| `build_design()` | `build_design()` | map samples to the four cells |
| `classify_responses()` | `classify_responses()` | the decision tree |
| `classification_options()` | `classification_options()` | presets and options |
| `model_definitions()` | `model_definitions()` | table of models and cases |
| `summarize_classification()` | `summarize_classification()` | counts and percentages |
| `primed_asymmetry()` | `primed_asymmetry()` | direction test for stress-ready states |
| `equivalence_test()` | `equivalence_test()` | TOST for log2 fold changes |
| `read_contrasts()` / `write_contrasts()` | `same` | contrast tables from / to files |
| `reciprocal_best_hits()` | `reciprocal_best_hits()` | orthologs from two BLAST files |
| `merge_ortholog_counts()` | `merge_ortholog_counts()` | ortholog-level count matrix |
| `plotting.plot_model_patterns()` | `plot_model_patterns()` | the fifteen patterns with counts |
| `plotting.plot_summary()` | `plot_summary()` | bar chart of models |
| `save_plots()` | `save_plots()` | write both figures |
| `OrthoResponseResult.save()` | `save_orthoresponse()` | write all outputs, including figures |
| `load_example()` | `load_example()` | bundled data (`rice_salt`, `ovarian_brequinar`) |
| `example_files()` / `export_example()` | `same` | paths / copies of the example files |

Every function is documented in place. In Python use, for example, `help(orr.classify_responses)`; in R, `?classify_responses`.

## 14 Troubleshooting

| message | cause and fix |
|---|---|
| *Empty design cell(s)* | `group_a`, `group_b`, `control` and `stress` do not match the values of the sample table (they are case-sensitive). |
| *Expected N fields in line M* | the sample table has spaces instead of tabs; fix it or read it with `sep=r"\s+"`. |
| *Counts are not integers* | TPM, FPKM or normalized values were given; use raw counts, or `round_counts=True` for estimated counts. |
| *Samples missing from the count matrix* | sample-table row names differ from the count-matrix columns. |
| *Contrast tables cover different features* | the five tables were filtered differently; only shared features are used. |
| *equivalence_margin requires an lfcSE column* | keep the standard-error column when exporting contrasts. |
| *run_contrasts() needs PyDESeq2* | `pip install "orthoresponse[deseq]"`. |
| No figures in the results folder | update to version 0.4.0 or later, and install matplotlib (`pip install "orthoresponse[plot]"`). |

## 15 Citation and license

If you use **orthoresponse**, please cite the software using the metadata in `CITATION.cff` (GitHub's *Cite this repository* button). DESeq2 and PyDESeq2 should be cited as well:

- Love MI, Huber W, Anders S (2014). Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. *Genome Biology* 15: 550.
- Muzellec B, Telenczuk M, Cabeli V, Andreux M (2023). PyDESeq2: a python package for bulk RNA-seq differential expression analysis. *Bioinformatics* 39: btad547.

**Data sources of the examples:** GEO GSE167342 (rice) and GEO GSE283381 (ovarian cancer cells).

**License:** MIT. **Source code:** github.com/Cold7/orthoresponse